

ICPC Bootcamp Day 2 Summary

Based on `day2_slides.pptx`

Session Summary

Day 2 studies how to navigate a search space. The opening slides define a search space as the complete set of candidate solutions to a problem. Some spaces are discrete, such as permutations, subsets, or board configurations. Others are continuous, such as a real-valued optimum in a geometry or physics problem. Competitive programming is often about describing this space precisely and then choosing a method that explores it fast enough.

The first main technique is complete search. Iterative complete search uses loops, nested loops, permutations, subsets, and pruning conditions to try possibilities directly. This approach is useful when the number of candidates is small or when constraints make the brute-force space manageable. The slides emphasize short-circuit boolean logic, `break`, and `continue` as simple pruning tools. Bitmask subset iteration extends Day 1's bitmask material: an integer can encode a subset, and submasks can be enumerated efficiently with bit operations instead of generating all 2^N masks and filtering.

The recursion review prepares students for recursive complete search, also called backtracking. A recursive search needs a clear base case, a recursive step, and a state that moves closer to termination. The 8-Queens example models the board one column at a time: try a row, check whether the placement is safe, recurse to the next column, and undo the placement when returning. This illustrates the difference between filtering and generating. Filtering builds many complete candidates and then rejects bad ones. Generating builds partial solutions and stops as soon as the partial state cannot lead to a valid solution.

The complete search tips give students practical ways to shrink search. Prune infeasible or inferior branches early. Use symmetries only when the benefit is large enough to justify extra code complexity. Precompute small fixed answer sets when queries repeat. Try solving a problem backwards when the forward view is too expensive. Compress data when constraints contain hidden structure, such as many repeated values or a small universe of distinct values. Optimize implementation details when necessary, but prefer a better algorithm over micro-optimization.

The second main technique is divide and conquer search, especially binary search. The deck stresses the monotonicity requirement: binary search is valid only when a yes/no condition changes direction once across the ordered search space. This supports standard lookup operations and "binary search the answer" problems such as minimizing the maximum feasible value or maximizing the minimum feasible value. Students are encouraged to use library functions such as Python's `bisect_left` and `bisect_right` for ordinary sorted-array searches.

The session closes with unimodal functions and ternary search. A unimodal function has one peak or one valley, so two interior test points can discard one third of the search interval at a time. Ternary search is therefore appropriate for optimization over a unimodal discrete or continuous domain, not for arbitrary functions.

Key Takeaways

- Define the state space before choosing complete search, backtracking, binary search, or ternary search.
- Backtracking is complete search with early rejection of invalid partial solutions.
- Binary search requires monotonicity; ternary search requires unimodality.
- Data compression, precomputation, and reversing the viewpoint can turn an impossible search into a feasible one.

CP4 Reading Connections

- CP4 Section 3.2, *Complete Search*, especially Sections 3.2.1, 3.2.2, and 3.2.3.
- CP4 Section 2.2.3, *Bitmask*, for subset and submask techniques.
- CP4 Section 3.3, *Divide and Conquer*, especially Section 3.3.1 on binary search usage and Section 3.3.2 on ternary search.
- CP4 Section 1.3.3 for checking whether a search strategy fits the input constraints.